

Practice - Week 2

1. What is UML?

- UML (Unified Modeling Language) là một ngôn ngữ mô hình hóa tiêu chuẩn được sử dụng trong kỹ thuật phần mềm để trực quan hóa, đặc tả, thiết kế và tài liệu hóa các hệ thống phần mềm. UML giúp biểu diễn cấu trúc và hành vi của hệ thống thông qua nhiều loại sơ đồ khác nhau.
- UML thường được sử dụng trong **(OOAD)** (Phân tích và thiết kế hướng đối tượng) để mô hình hóa cách các thành phần trong hệ thống tương tác với nhau.

2. Why is UML important?

- **Chuẩn hoá giao tiếp giữa các bên**
 - Developer: *quan tâm class, API, service*
 - Data Scientist: *quan tâm model & data flow*
 - MLOps Engineer: *quan tâm deployment & infrastructure*
 - Business Analyst: *quan tâm use case*
 - Stakeholder
- **Giảm rủi ro trước khi code**
 - Phát hiện thiếu requirement
 - Phát hiện dependency phức tạp
 - Tránh circular dependency
 - Phát hiện bottleneck kiến trúc

→ Sửa lỗi ở giai đoạn design rẻ hơn rất nhiều so với sửa khi đã triển khai production.
- **Hỗ trợ tư duy hệ thống (System Thinking)**
 - AI system không chỉ là một mô hình ML.
 - Một hệ thống thực tế bao gồm:

- Data ingestion
- Data preprocessing
- Feature engineering
- Model training
- Model serving
- Monitoring
- Feedback loop
- Retraining
- UML giúp:
 - Nhìn toàn bộ hệ thống thay vì chỉ model
 - Xác định rõ boundary giữa các module
 - Thiết kế pipeline rõ ràng
- **Tài liệu hoá hệ thống (Documentation)**
 - Trong thực tế:
 - Nhân sự thay đổi
 - Model được nâng cấp
 - Kiến trúc phát triển theo thời gian
 - Nếu không có tài liệu thiết kế:
 - Người mới mất nhiều thời gian hiểu hệ thống
 - Việc refactor rất rủi ro
 - Audit AI system khó khăn
 - UML đóng vai trò:
 - Tài liệu kỹ thuật chính thức
 - Bản đồ kiến trúc hệ thống
 - Cơ sở cho bảo trì và mở rộng
- **Tách biệt rõ Design và Implementation**

- Bắt đầu code ngay mà chưa thiết kế.
- UML giúp:
 - Tập trung vào abstraction
 - Không bị lệ thuộc framework
 - Không bị ràng buộc bởi ngôn ngữ lập trình
- Design tốt có thể:
 - Triển khai bằng Python
 - Hoặc Java
 - Hoặc microservices
- Design không nên phụ thuộc implementation.
- **Khi nào không cần UML?**
 - Không cần UML khi:
 - Prototype nhanh
 - Project nhỏ
 - Một người phát triển
 - Nghiên cứu thử nghiệm (research code)
 - cần UML khi:
 - Hệ thống production
 - Có nhiều người tham gia
 - Có yêu cầu bảo trì lâu dài
 - Có deployment & monitoring

3. What are UML Diagrams?

- UML có nhiều loại sơ đồ khác nhau, được chia thành hai nhóm chính:
 - Structural Diagrams (Sơ đồ cấu trúc): Mô tả cấu trúc tĩnh của hệ thống.
 - Behavioral Diagrams (Sơ đồ hành vi): Mô tả cách hệ thống hoạt động theo thời gian.

- Vì mọi hệ thống đều có hai khía cạnh cơ bản → phản ánh đúng cách con người tư duy
 - Hệ thống là gì?
 - Những thành phần nào?
 - Quan hệ giữa chúng là gì?
 - Ai phụ thuộc vào ai?
 - Hệ thống hoạt động như thế nào?
 - Khi có sự kiện xảy ra thì điều gì diễn ra tiếp theo?
 - Luồng xử lý thế nào?
 - Trạng thái thay đổi ra sao theo thời gian?

4. Structural Diagrams

- Class Diagram (Sơ đồ lớp)
 - **Mô tả:** Biểu diễn các lớp trong hệ thống, thuộc tính, phương thức và mối quan hệ giữa chúng.
 - **Ứng dụng:** Dùng để thiết kế cơ sở dữ liệu và mô hình hóa logic của hệ thống.
 - **Trong tài liệu:**
 - Mô tả quan hệ giữa Khách hàng, Tài khoản, Giỏ hàng, Đơn hàng.
 - Mỗi khách hàng có một tài khoản duy nhất.
 - Giỏ hàng có thể chứa nhiều sản phẩm, và đơn hàng có thể có nhiều thanh toán.
- Object Diagram (Sơ đồ đối tượng)
 - **Mô tả:** Biểu diễn một "snapshot" (trạng thái cụ thể) của hệ thống tại một thời điểm nhất định.
 - **Ứng dụng:** Dùng để kiểm tra xem các đối tượng thực tế sẽ tương tác như thế nào trong hệ thống.
 - **Trong tài liệu:** Mô tả trạng thái của các đối tượng liên quan đến quá trình đăng nhập người dùng, gồm Login Controller, User Manager,

Cookie Manager, Logger.

- Component Diagram (Sơ đồ thành phần)
 - **Mô tả:** Biểu diễn các thành phần phần mềm và cách chúng giao tiếp với nhau.
 - **Ứng dụng:** Dùng để hiểu cách các module phần mềm hoạt động cùng nhau.
 - **Trong tài liệu:** Mô tả các thành phần của hệ thống mua sắm, bao gồm:
 - Công cụ tìm kiếm (Search Engine)
 - Giỏ hàng (Shopping Cart)
 - Xác thực người dùng (Authentication)
 - Quản lý đơn hàng (Orders)
- Deployment Diagram (Sơ đồ triển khai)
 - **Mô tả:** Biểu diễn cách hệ thống phần mềm được triển khai trên phần cứng.
 - **Ứng dụng:** Dùng để mô tả kiến trúc triển khai trên server.
 - **Trong tài liệu:** Hệ thống được triển khai trên máy chủ Apache Tomcat, bao gồm các thành phần như:
 - Database Server
 - Application Server
 - Web Server
- Others

5. Behavioral Diagrams

- Use Case Diagram (Sơ đồ ca sử dụng)
 - **Mô tả:** Biểu diễn các tác nhân bên ngoài (users, hệ thống khác) và cách chúng tương tác với hệ thống.
 - **Ứng dụng:** Dùng để xác định chức năng chính của hệ thống.
 - **Trong tài liệu:**

- Người dùng có thể xem sản phẩm, đăng ký tài khoản, mua hàng.
- Khi mua hàng, họ cần phải đăng nhập, thêm sản phẩm vào giỏ hàng, thanh toán.
- Sequence Diagram (Sơ đồ trình tự)
 - **Mô tả:** Biểu diễn trình tự các bước trong một quy trình.
 - **Ứng dụng:** Dùng để hiểu cách các đối tượng tương tác với nhau theo thời gian.
 - **Trong tài liệu:** Khi người dùng mua sách, họ sẽ tìm kiếm, xem mô tả, thêm vào giỏ hàng, thanh toán.
- Communication Diagram (Sơ đồ giao tiếp)
 - **Mô tả:** Biểu diễn cách các đối tượng liên kết và trao đổi dữ liệu với nhau.
 - **Ứng dụng:** Dùng để hiểu cách các thành phần của hệ thống tương tác với nhau.
 - **Trong tài liệu:** Mô tả cách khách hàng có thể tìm kiếm sách, xem mô tả, thêm vào giỏ hàng.
- Activity Diagram (Sơ đồ hoạt động)
 - **Mô tả:** Biểu diễn quy trình làm việc trong hệ thống.
 - **Ứng dụng:** Dùng để mô tả luồng xử lý trong hệ thống.
 - **Trong tài liệu:** Mô tả quy trình tìm kiếm, xem sản phẩm, thêm vào giỏ hàng, thanh toán.
- Interaction Overview Diagram (Sơ đồ tổng quan tương tác)
 - **Mô tả:** Tổng hợp các sơ đồ tương tác.
 - **Ứng dụng:** Dùng để có cái nhìn tổng thể về tất cả các tương tác trong hệ thống.
 - **Trong tài liệu:** Tổng hợp tìm kiếm sản phẩm, thêm vào giỏ hàng, thanh toán.
- Others